SAVEETHA SCHOOL OF ENGINEERING

OPERATING SYSTEMS

ASSIGNMENT | CO1

EVALUATE AND JUSTIFY THE OPERATING-SYSTEM DESIGN OF A SMART HOSPITAL

Student Details	Information
Name	S. Maruthu Pandi
Register Number	192511162
Department	B.E. Computer Science and Engineering (Cyber Security)
Course	CSA0421 – Operating Systems
Assignment	Assignment 1 (Tasks 2)

TASK 2 – JUDGE THE TYPE OF OPERATING SYSTEM FOR EACH ENVIRONMENT

2.1 Relevant operating-system types

OS type	Main characteristic	Suitability in the hospital
Mainframe	Large, centralized, multi-user system with strong I/O and transaction handling.	Useful as a conceptual model for centralized record processing, but not required for every hospital workload. A modern clustered server is a more flexible fit for this scenario.
Desktop	Interactive, general-purpose OS with GUI, drivers and broad application support.	Best for doctors' workstations; it can also support nurse applications but is less power-focused than a handheld platform.
Multiprocessor	One computer contains multiple CPUs/cores sharing memory and I/O.	Best for MRI processing and can also run database workloads. It increases compute capacity inside one node but does not by itself remove a single point of failure.
Distributed	Independent networked computers cooperate and exchange data.	Required for sharing patient records among the three hospitals and for future cloud services.
Clustered	Multiple connected nodes operate as one service or coordinated group, often with fail-over.	Best for the central patient database because another node can take over when a node fails.
Hard real-time	Every critical task must meet its deadline; missing it is a system failure.	Best for the bedside alarm path because the 10 ms deadline is safety-critical.

OS type	Main characteristic	Suitability in the hospital
Soft real-time	Deadlines matter, but occasional misses reduce quality rather than causing immediate catastrophic failure.	Appropriate for image-processing queues, video or non-critical clinical displays.
Handheld / mobile	Optimized for touch, wireless connectivity, low power and constrained resources.	Best for nurses' tablets.

2.2 Recommended type for each hospital environment

Environment	Recommended type	Rejected alternative and reason
Patient monitors	Embedded hard-real-time system. The monitor samples sensors and activates alarms locally.	Desktop or time-sharing OS: average responsiveness is not a proof of a worst-case 10 ms response.
Nurses' tablets	Handheld/mobile OS with managed enterprise controls.	Full workstation OS: unnecessary services and higher power/storage costs.
Doctors' workstations	Desktop multi-user capable OS.	Embedded RTOS: poor fit for rich GUI, broad application compatibility and interactive multitasking.
Patient database	Clustered server OS with fail-over, preferably on multiple physical nodes.	One unclustered server: a hardware or kernel failure can stop all record access.
MRI imaging	SMP multiprocessor server OS with high-throughput storage and DMA.	Single-core device: insufficient parallel compute capacity for large image workloads.

Environment	Recommended type	Rejected alternative and reason
Partner hospitals	Distributed-system communication across secured networks.	One isolated local server: cannot provide shared records across sites.

2.3 Symmetric versus asymmetric multiprocessing for MRI

Symmetric multiprocessing (SMP) allows all processors to run operating-system and application tasks and to access shared memory under a common OS instance. It is the preferred option for the imaging server because a parallel image-reconstruction application can distribute work across available CPU cores. The OS scheduler balances runnable threads, and the application can use threads or a parallel processing framework.

Asymmetric multiprocessing (AMP) assigns different processors fixed or specialized roles. For example, one processor might handle acquisition while another performs reconstruction. AMP can be useful in a tightly partitioned embedded design, but it adds coordination and may leave capacity unused when workloads are uneven. For a general MRI image server, SMP is more flexible. A dedicated acquisition controller can still be used alongside the SMP host.

2.4 Clustered database with fail-over

Use at least two database nodes with a third quorum witness or an equivalent consensus mechanism. One node may be active and another ready to take over, or the database product may support a carefully configured active-active arrangement. Health checks detect a failed node; the cluster manager fences the failed node to prevent two nodes writing independently; a healthy node is promoted; and clients reconnect through a stable service address. Storage replication, transaction-log recovery and backup restoration must be tested.

Fail-over within seconds is a design target, not an automatic guarantee. The measured recovery time depends on failure detection, fencing, database recovery, network routing and client retry behaviour. Clinical applications should show a clear connection state and avoid silently duplicating transactions.

2.5 Alternative: one large multiprocessor database machine

Factor	Single large SMP machine	Cluster of smaller fail-over nodes
Availability	The machine is a single point of failure. CPU, memory, motherboard, power or OS failure can stop the database.	A node failure can be handled by promotion of a healthy node if quorum, replication and fencing are correct.
Cost	Can be expensive due to large-memory systems, high-end CPUs and redundant components inside one chassis.	Requires multiple nodes and cluster/network design, but commodity nodes may be easier to expand and replace.
Throughput as processors increase	Improves while parallel work exists, but serial code and shared memory/I/O contention limit gains.	Capacity can be increased vertically per node and, depending on database architecture, horizontally across nodes or replicas.
Memory capacity	A large server may support substantial RAM, giving a large cache, but all memory is lost to the service if the host fails.	Memory is distributed across nodes; a fail-over node must have enough RAM to serve the workload after promotion.

A useful upper-bound model is Amdahl's law: Speedup(N) = 1 / ((1 − P) + P/N), where P is the parallel fraction and N is the processor count. If 90% of a workload is parallel, the theoretical speed-up with 8 processors is 1 / (0.1 + 0.9/8) ≈ 4.71, not 8. Real systems can be slower because of locking, memory bandwidth and I/O. Adding processors does not automatically add memory capacity; RAM must be sized to the working set and cache requirements. The cluster is selected for availability, while SMP improves throughput within each node.